Comparación de Algoritmos de Búsqueda de Caminos

Tarea 1 — Inteligencia Artificial

Javier Molina

Ingeniería en Desarrollo de Videojuegos y Realidad Virtual

1. Introducción

En el contexto de la IA para Videojuegos, la búsqueda de caminos es fundamental para simular comportamientos realistas de agentes en entornos dinámicos. Esta tarea busca demostrar competencias de la implementación y comparación de 4 algoritmos: Breadth-Firts Search (BFS), Greedy Best-First Search (Greedy BFS), A* y Weighted A*(WA), evaluando métricas como el tiempo de ejecución, longitud de camino y nodos revisados en mapas binarios y de alturas proporcionados.

2. Algoritmos Implementados

2.1 Breadth-First Search (BFS)

BFS va nivel por nivel: chequea todos los puntos cercanos antes de ir más lejos. Encuentra la ruta más corta en mapas sin distancias complicadas, pero revisa todo el mapa, así que es lento en lugares grandes.

2.2 Greedy Best-First Search

Este va directo al objetivo usando solo una "pista" para elegir el siguiente paso. Es el más rápido, pero ignora lo que ya recorrió, así que a veces da rutas no ideales.

2.3 A-Star (A^*)

Mezcla lo que ya recorriste con la pista al objetivo. Si la pista es confiable, siempre da la mejor ruta y chequea mucho menos que BFS

2.4 Weighted A-Star (WA*)

Como A*, pero le da más peso a la pista (multiplicada por 2 en este caso). Explora poquísimo, pero la ruta puede ser un poco más larga, aunque controlada.

3. Mapas Utilizados

Se evaluaron cuatro mapas de gran escala (320×512 celdas), representativos de escenarios reales en videojuegos:

3.1 random_big_map (Mapa de Alturas)

Cada celda tiene una altura al azar (0-9). Todo es caminable, pero moverte cuesta según la diferencia de altura. Es un enredo sin forma fija.

3.2 perlin_big_map (Mapa de Alturas)

Similar al anterior, pero con alturas generadas por Perlin Noise: colinas y valles naturales. Ayuda a los algoritmos "inteligentes" porque el terreno guía hacia el objetivo..

3.3 binary_maze_big_map (Mapa Binario)

Laberinto puro (0=libre, 1=muro) hecho con backtracker. Pasillos angostos y muchas vueltas, obliga a explorar un mucho.

3.4 binary_obstacle_big_map (Mapa Binario)

30% de celdas bloqueadas al azar (0=libre, 1=muro). Espacio abierto y conectado, así que los algoritmos listos lo resuelven rápido.

3.5 Diseño Experimental Y Uso del Programa

Corrimos cada algoritmo 30 veces por mapa para promediar bien. Medimos tiempo (en ms), largo de la ruta y nodos chequeados, para usar el programar debe seleccionar uno de los algoritmos o la opción de todos y

4. Resultados Experimentales

A continuación se presentan los resultados promedio de 30 ejecuciones por algoritmo en cada mapa. Las tablas incluyen tiempo de ejecución, largo del camino encontrado y nodos revisados.

Tabla 1. *random_big_map*
(alturas aleatorias, 320×512)

Algoritmo	Tiempo (ms)	Largo Camino	Nodos Revisados
BFS	56.3127	1 069	163 840
Greedy BFS	1.1170	1 645	512
A^* (w=1,0)	169.0785	477	115 756
WA* (w=2,0)	3.0677	733	767

Tabla 2. *perlin_big_map*
(alturas Perlin, 320×512)

Algoritmo	Tiempo (ms)	Largo Camino	Nodos Revisados
BFS	58.4687	1 069	163 840
Greedy BFS	1.0690	277	512
A^* (w=1,0)	2.5823	259	679
WA* (w=2,0)	2.3327	267	512

Tabla 3. *binary_maze_big_map*
(laberinto binario, 320×512)

Algoritmo	Tiempo (ms)	Largo Camino	Nodos Revisados
BFS	17.9764	11 711,63	60 590
Greedy BFS	17.3568	11 741,15	20 531
A^* (w=1,0)	35.3702	11 711,63	59 528
WA* (w=2,0)	35.4937	11 713,27	57 472

Tabla 4. *binary_obstacle_big_map*
(obstáculos dispersos, 320×512)

Algoritmo	Tiempo (ms)	Largo Camino	Nodos Revisados
BFS	41.5155	664,52	114 614
Greedy BFS	0.9712	691,21	547
A^* (w=1,0)	31.0140	645,92	50 406
WA* (w=2,0)	2.1892	681,73	553

5. Discusión y Conclusiones

Al analizar los resultados de las cuatro tablas se confirman las propiedades teóricas de cada algoritmo y emergen patrones claros en función del tipo de mapa.

5.1 BFS

Como era esperado, BFS revisó consistentemente la mayor cantidad de nodos en todos los escenarios, con tiempos de ejecución que escalan directamente con el tamaño del mapa. Sin embargo, su garantía de optimalidad en mapas binarios queda evidenciada en la Tabla 3, donde entregó el mismo largo de camino óptimo que (11 711,63).

5.2 Greedy BFS

Greedy BFS resultó consistentemente el algoritmo más rapido, llegando a resolver el mapa de obstáculos en menos de 1 ms. Sin embargo, la ausencia de $g(n)$ en su función de evaluación se traduce en rutas subóptimas en el mapa random entregó un camino de largo 1 645 frente al óptimo de 477 de A^*, más del triple.

5.3 A-Star (A^*)

demostró el mejor equilibrio entre calidad de solución y exploración. En el mapa Perlin encontró el camino más corto de todos los algoritmos (largo 259) con solo 679 nodos. Su principal desventaja aparece en el mapa random, donde el terreno desordenado desorientó la heurística y obligó a explorar 115 756 nodos con un tiempo de 169 ms.

5.4 Weighted A-Star (WA*)

WA* con $w = 2{,}0$ se perfiló como la alternativa más pragmática para videojuegos: en todos los mapas mantuvo tiempos bajo 36 ms y nodos revisados drásticamente inferiores a A^*, sacrificando un margen acotado de optimilidad. En el mapa de obstáculos la degradación fue mínima (681,73 vs 645,92), con una ganancia de velocidad de 14 veces.

5.5 Conclusión General

La elección del algoritmo debe considerar las restricciones del contexto de aplicación. En videojuegos con frame rate limitado, WA* resulta la opción más equilibrada para NPCs en mapas abiertos. A^* es preferible cuando la optimalidad es crítica y el mapa posee estructura geográfica suave, como terrenos generados con Perlin Noise. BFS queda reservado para grafos pequeños o cuando se requiere una exploración completa.

El laberinto evidenció que ningún algoritmo heurístico tiene ventaja en espacios altamente encerrados, en ese escenario, la topología supera a la heurística y los tiempos convergen entre todos los algoritmos.

Repositorio del Proyecto

https://github.com/Jarkeer/Tarea-1-IA